MySQL & PostgreSQL Data Types

Complete Quick Revision Cheat Sheet for Computer Science & Engineering Students

Section 1: Introduction

What is a Data Type?

A data type is an attribute that tells the database engine what kind of value a column can store (e.g., integers, text, dates).

Why are they important?

They guarantee data integrity, optimize disk storage utilization, and speed up query performance by allocating precise memory space.

MySQL vs. PostgreSQL

MySQL focuses on speed and simplicity with legacy web types, whereas PostgreSQL offers enterprise-level flexibility, strict standards, and advanced structures like arrays.

Section 2: MySQL Data Types Cheat Sheet

Category	Data Type	Size	Description (One-Sentence)	Example & Use Case
Numeric Types	`TINYINT`	1 Byte	Stores very small whole numbers from -128 to 127 (or 0 to 255 unsigned).	`12` (User age)
	`SMALLINT`	2 Bytes	Stores small integers ranging from -32,768 to 32,767.	`350` (Page count)
	`MEDIUMINT`	3 Bytes	Stores medium-sized integers from -8,388,608 to 8,388,607.	`150000` (City population)
	`INT` / `INTEGER`	4 Bytes	Stores standard whole numbers up to ~2.1 billion.	`742011` (User ID)
	`BIGINT`	8 Bytes	Stores extremely large whole numbers for massive datasets.	`9223372036` (Global transaction ID)
	`DECIMAL(P,D)`	Fixed	Stores exact fixed-point numbers where precision (P) and scale (D) are strictly preserved.	`19.99` (Product price)
	`FLOAT`	4 Bytes	Stores approximate floating-point single-precision values.	`72.45` (Body weight)
	`DOUBLE`	8 Bytes	Stores high-precision approximate double-precision values.	`3.14159265` (Scientific calculation)
	`BIT(M)`	1-8 Bytes	Stores a collection of binary bit values up to length M.	`b'101'` (Feature flags)
String Types	`CHAR(M)`	Fixed	Stores fixed-length character strings right-padded with spaces up to length M.	`'NY'` (State abbreviations)
	`VARCHAR(M)`	Variable	Stores variable-length text strings up to a maximum length of M characters.	`'john_doe'` (Usernames)
	`TINYTEXT`	Length+1B	Stores tiny non-binary character strings up to 255 characters.	`'Short summary'` (Brief card note)
	`TEXT`	Length+2B	Stores standard length body text strings up to 65,535 characters.	`'Full blog text...'` (Article posts)

Category	Data Type	Size	Description (One-Sentence)	Example & Use Case
	`MEDIUMTEXT`	Length+3B	Stores medium-length character strings up to 16.7 million characters.	`'HTML source code'` (Webpage logs)
	`LONGTEXT`	Length+4B	Stores massive long-form text files up to 4.29 gigabytes.	`'Entire novel text'` (E-books archives)
Date / Time	`DATE`	3 Bytes	Stores a simple calendar date value in `YYYY-MM-DD` format.	`'2026-06-20'` (Date of birth)
	`TIME`	3 Bytes	Stores a time duration or clock time value in `HH:MM:SS` format.	`'14:30:00'` (Class start time)
	`DATETIME`	8 Bytes	Stores a combined static date and time value independent of time zones.	`'2026-06-20 09:15:00'` (Booking time)
	`TIMESTAMP`	4 Bytes	Stores automatic time zone-converted Unix era epoch timestamp values.	`'2026-06-20 03:45:00 UTC'` (System logs)
	`YEAR`	1 Byte	Stores a simple four-digit year value from 1901 to 2155.	`2026` (Graduation year)
Binary Types	`BINARY(M)`	Fixed	Stores fixed-length raw byte strings padding empty spaces with zeroes.	`0x4F3E` (Encrypted keys)
	`VARBINARY(M)`	Variable	Stores variable-length raw byte streams up to size length M.	`0x7A8B9C` (Password hashes)
	`TINYBLOB`	Length+1B	Stores tiny raw binary large objects up to 255 bytes max.	`Small binary file` (Icon graphics)
	`BLOB`	Length+2B	Stores basic binary large objects up to 65,535 bytes.	`Image byte stream` (User profile photo)
	`MEDIUMBLOB`	Length+3B	Stores medium binary files up to 16.7 megabytes.	`Audio recording` (Short voice memo)
	`LONGBLOB`	Length+4B	Stores immense binary files up to 4.29 gigabytes in size.	`Video file` (Lecture video archive)
Special Types	`ENUM`	1-2 Bytes	Stores exactly one string chosen from a predefined custom static list.	`'Low', 'Medium', 'High'` (Priority)
	`SET`	1-8 Bytes	Stores zero or multiple string elements selected from a predefined collection.	`'Math, Physics'` (Enrolled courses)
	`JSON`	Variable	Stores and enforces validated native JSON text documents natively.	`{"dark_mode": true}` (App settings)

Section 3: PostgreSQL Data Types Cheat Sheet

Category	Data Type	Size	Description (One-Sentence)	Example & Use Case
Numeric Types	`SMALLINT`	2 Bytes	Stores small integers ranging from -32,768 to 32,767.	`120` (Room number)
	`INTEGER`	4 Bytes	Stores standard corporate scale database integers up to 2.1 billion.	`52311` (Employee ID)

Category	Data Type	Size	Description (One-Sentence)	Example & Use Case
	`BIGINT`	8 Bytes	Stores massive long-integer numbers up to 9 quintillion.	`9876543210` (Global phone number)
	`DECIMAL` / `NUMERIC`	Variable	Stores user-defined user-specified exact numeric scales up to 131,072 digits.	`12500.50` (Financial accounts)
	`REAL`	4 Bytes	Stores variable fractional inexact precision real numbers (6 decimal digits).	`23.45` (Sensor temperature)
	`DOUBLE PRECISION`	8 Bytes	Stores variable inexact floating-point numbers (15 decimal digits).	`45.123456789` (GPS Coordinates)
	`SERIAL`	4 Bytes	Creates an auto-incrementing integer identifier column from 1 to 2.1 billion.	`1, 2, 3...` (Auto-assigned primary key)
	`BIGSERIAL`	8 Bytes	Creates an auto-incrementing massive 8-byte big integer row identifier.	`1000000001` (Large scale table key)
	`MONEY`	8 Bytes	Stores absolute currency amounts with localized currency formatting symbols.	`$550.25` (Product pricing)
Character Types	`CHAR(n)`	Fixed	Stores blank-padded character strings exactly matching length n.	`'USA'` (Country ISO codes)
	`VARCHAR(n)`	Variable	Stores flexible non-blank-padded character strings limited to length n.	`'Jane Smith'` (Customer names)
	`TEXT`	Variable	Stores limitless length character text strings without any length penalty.	`'Product reviews...'` (Unlimited comment field)
Date / Time	`DATE`	4 Bytes	Stores calendar date specifications (year, month, day).	`'2026-12-31'` (Project deadline)
	`TIME`	8 Bytes	Stores precise time of day clocks without time zone context.	`'09:00:00'` (Office opening hour)
	`TIMESTAMP`	8 Bytes	Stores combined static calendar date and time clock instances.	`'2026-06-20 14:00:00'` (Flight arrival)
	`TIMESTAMPTZ`	8 Bytes	Stores fully active time zone aware timestamp records safely.	`'2026-06-20 14:00:00+05:30'` (International order)
	`INTERVAL`	16 Bytes	Stores calculated mathematical time spans and historical durations.	`'3 days 2 hours'` (Trial subscription lifespan)
Boolean Type	`BOOLEAN`	1 Byte	Stores precise logical boolean values representing true, false, or NULL states.	`TRUE` (Is email verified)
Binary Type	`BYTEA`	Variable	Stores raw binary data files directly into an editable byte array string.	`A4243` (PDF file attachments or blobs)
JSON Types	`JSON`	Variable	Stores raw exact text copies of JSON files verifying valid formatting syntax.	`{"id": 5}` (Order log metadata)
	`JSONB`	Variable	Stores decompressed binary JSON structures supporting rapid background indexing.	`{"roles": ["admin"]}` (Fast queried profile rules)

Category	Data Type	Size	Description (One-Sentence)	Example & Use Case
UUID Type	`UUID`	16 Bytes	Stores globally unique system identifiers guaranteed unique across networks.	`'a0eebc99-9c0b...'` (Microservices key)
Network Types	`INET`	7 or 19 B	Stores individual host IPv4 or IPv6 system web connection addresses.	`'192.168.1.1'` (Client computer IP)
	`CIDR`	7 or 19 B	Stores complete subnet corporate IP network blocks safely.	`'10.0.0.0/8'` (Corporate network block)
	`MACADDR`	6 Bytes	Stores low-level physical computer hardware interface media access keys.	`'08:00:2b:01:02:03'` (Network router ID)
Array Types	`INTEGER[]`	Variable	Stores ordered multi-dimensional lists of generic integer items.	`'{95, 88, 100}'` (Student class grades)
	`TEXT[]`	Variable	Stores organized collections or tag lists of text string values.	`'{"sql", "db"}'` (Article content tags)
Special Type	`XML`	Variable	Validates and holds well-formed enterprise legacy XML schema documents.	`'<user>admin</user>'` (Legacy integration data)

Section 4: MySQL vs PostgreSQL Data Types Comparison

Feature Group	MySQL	PostgreSQL
Integer Auto-Increment	Uses the keyword `AUTO_INCREMENT` modifier on normal integers.	Uses clean pseudo-types `SERIAL` and `BIGSERIAL`.
Unbounded Text Fields	Requires structural size tags like `TEXT` or `LONGTEXT`.	Allows infinite length `TEXT` fields natively without any penalty.
Time Zone Handling	Offers distinct types: `DATETIME` (fixed) and `TIMESTAMP` (TZ-aware).	Explicitly splits fields cleanly using `TIMESTAMP` and `TIMESTAMPTZ`.
JSON Architecture	Supports standard functional `JSON` document columns.	Provides superior `JSONB` binary format for super-fast indexed queries.
UUID Framework	No native type; requires manual workarounds using `VARCHAR(36)`.	Features a dedicated native 16-byte performance-optimized `UUID` type.
Array Data Support	Does not support arrays; requires building messy join tables.	Supports native arrays on all core types (e.g., `TEXT[]`).
Network Addresses	Must store network IPs as plain text string identifiers.	Features dedicated system types: `INET`, `CIDR`, `MACADDR`.
Performance Focus	Optimized for speed and rapid reads in typical web applications.	Optimized for high-concurrency complex analysis and enterprise operations.
Flexibility & Extensibility	Provides a rigid set of built-in web-focused options.	Allows users to create custom types using `CREATE TYPE` easily.

Section 5: Most Important Exam Questions (Short Answers)

Q1: What are Data Types?

Ans: Data Types are attributes that define what kind of data an element can hold (such as text, integers, or dates), telling the database engine how much storage space to allocate and what mathematical actions are allowed on that column.

Q2: Explain Numeric Data Types briefly.

Ans: Numeric types hold number configurations split into **Exact Numbers** like `INT` and `DECIMAL` (crucial for exact values like financial accounts) and **Approximate Numbers** like `FLOAT` and `DOUBLE` (used for broad scientific estimations).

Q3: Explain Character Data Types.

Ans: Character types store alpha-numeric textual values and sentences using short fixed lengths (`CHAR`), dynamic bounded variables (`VARCHAR`), or massive unrestricted text blocks (`TEXT`).

Q4: Explain Date and Time Data Types.

Ans: These types store chronographic records; common variations include standalone dates (`DATE`), simple hours (`TIME`), and complete calendar snapshots containing dynamic time-zone data offsets (`TIMESTAMP`).

Q5: What is the core difference between CHAR and VARCHAR?

Ans: `CHAR` is a **fixed-length** type that pads empty characters with spaces to consume full disk space, while `VARCHAR` is a **variable-length** container allocating only the actual string length plus minimal metadata overhead.

Q6: What is the difference between JSON and JSONB in PostgreSQL?

Ans: The basic `JSON` type saves an exact, unparsed textual copy requiring re-parsing on every read, while `JSONB` stores data parsed into a compressed binary format that is slightly slower to insert but vastly faster to search and index.

Q7: Summarize the biggest difference between MySQL and PostgreSQL data types.

Ans: PostgreSQL natively supports advanced multi-value and structural types like `UUID`, IP addresses (`INET`), and custom arrays (`INT[]`), whereas MySQL relies primarily on standard web text/numeric scalars and requires workarounds for arrays or complex types.

Section 6: One-Page Quick Revision Sheet

Core Data Type	Database System	Primary Purpose / Quick Rule	Exam Example
`INT / INTEGER`	**MySQL** & **PostgreSQL**	Stores standard whole identification counts.	`452012`
`DECIMAL / NUMERIC`	**MySQL** & **PostgreSQL**	Ensures precise decimal math without rounding errors.	`99.45`
`VARCHAR(n)`	**MySQL** & **PostgreSQL**	Saves flexible text strings efficiently up to size limit n.	`'@Alex_123'`
`TEXT`	**MySQL** & **PostgreSQL**	Handles long-form dynamic descriptions or paragraphs.	`'User review here...'`
`TIMESTAMPTZ`	**PostgreSQL Only**	Guarantees robust international time-zone alignment.	`'2026-06-20 09:15-05'`
`SERIAL`	**PostgreSQL Only**	Generates simple auto-incrementing database primary keys.	`1, 2, 3...`
`JSONB`	**PostgreSQL Only**	Optimizes schema-less unstructured records with rapid indexing.	`{"status": "active"}`
`UUID`	**PostgreSQL Only**	Creates globally unique system distributed primary keys.	`'f81d4fae-7dec...'`
`TEXT[]`	**PostgreSQL Only**	Stores multi-value tag item sheets inside a single row.	`'{"cs", "it", "ee"}'`

Prepared as a Student Quick Revision Cheat Sheet for Database Management Systems (DBMS).